Programación concurrente y paralela

Práctica 6 - Mensajes

Segundo Cuatrimestre 2026



Do not communicate by sharing memory; instead, share memory by communicating.



Fundamentos de Go

Variables, inferencia y constantes

```go
var count int = 10
var total = 0       // int inferido
var ready bool      // false

func main() {
    name := "Mario Santos"
    count = 20      // mismo tipo
}
```

```go
const Pi = 3.14159       // sin tipo
const MaxUsers int = 100 // tipada

const (
    Pending = iota // 0
    Running        // 1
    Done           // 2
)
```

`var`: en cualquier scope

`:=`: solo dentro de funciones

Sin inicializador ⇒ valor cero

`const` ⇒ valor de compilación



Loops

```go
// Forma clasica
for i := 0; i < 5; i++ {
    fmt.Println(i)
}

// Como un while
for pending > 0 {
    foo()
    pending--
}
```

```go
// Recorrer una coleccion
for i, value := range values {
    if value < 0 {
        continue
    }
    fmt.Println(i, value)
}

for { // loop infinito
    break
}
```



Go solo tiene `for`

`range` produce índice y copia del valor

`break`: termina el loop

`continue`: avanza a la siguiente iteración

Funciones también tienen tipo

```go
func add(a, b int) int {
    return a + b
}

func div(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("zero")
    }
    return a / b, nil
}
```

```go
type Transform func(int) int

func apply(f Transform, x int) int {
    return f(x)
}

var double = func(x int) int {
    return 2 * x
}
var result = apply(double, 3)

func greeting() {
    defer fmt.Println("Chau.")
    fmt.Println("Hola!") // luego "Chau."
}
```

Parámetros y retornos tipados Múltiples retornos Funciones como valores

`defer` ejecuta la llamada al salir de la función; varios se resuelven en orden LIFO



Structs

```go
type User struct {
    Name   string
    Active bool
}

var nobody User // "", false
```

```go
ravenna := User{Name: "Emilio Ravenna"}

ravenna.Active = true

cozzetti := ravenna // copia del struct
cozzetti.Name = "Maximo Cozzetti"

fmt.Println(ravenna.Name)
```

Campos con tipo Valor cero utilizable Literales con campos nombrados

Los structs son valores: asignarlos o pasarlos crea una copia



Funciones vs. métodos

Función

```
1 func Activate(u *User) {
2     u.Active = true
3 }
4
5 Activate(&ravenna)
```

Todos los argumentos son explícitos.

Método

```
1 func (u *User) Activate() {
2     u.Active = true
3 }
4
5 ravenna.Activate()
```

`u *User` es el receiver.

Los métodos se declaran fuera del struct y se asocian a un tipo nombrado.

Receiver `T`: copia Receiver `*T`: puede modificar Métodos ⇒ interfaces

Interfaces

```go
type Greeter interface {
    Greet() string
}

func (u User) Greet() string {
    return "Hola, " + u.Name
}
```

```go
func Welcome(g Greeter) {
    fmt.Println(g.Greet())
}

var g Greeter = ravenna
Welcome(g)
```

User tiene el método requerido:

`Greet() string`

El compilador verifica

La asignación es válida porque el conjunto de métodos de `User` contiene `Greet() string`.

No existe `implements`: la satisfacción es implícita.

Firma exacta

Métodos adicionales permitidos

Interfaces pequeñas



Punteros

```go
func increment(x *int) {
    *x = *x + 1
}

n := 10
increment(&n)

fmt.Println(n) // 11
```

Como en C

&x obtiene la dirección

*p accede al valor

Diferencia clave

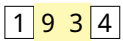
No hay aritmética de punteros

Arrays, slices y views

```go
array := [4]int{1, 2, 3, 4}
slice := array[:]     // len=4, cap=4
view  := slice[1:3]  // [2 3]

view[0] = 9
fmt.Println(array)   // [1 9 3 4]

// cap lleno: crea otro backing array
slice = append(slice, 5)
```



`view` comparte los elementos resaltados.

Array: `[N]T`

Tamaño fijo; `N` forma parte del tipo.
Sus elementos son modificables.
Asignarlo o pasarlo copia sus elementos.

Slice: `[]T`

Descriptor `{ptr, len, cap}`.
Sus elementos son modificables.
`append` puede cambiar el backing array.

View: `s[i:j]`

También es un slice: comparte memoria.
Modificarla puede afectar a `s` y al array.

Goroutines

```go
func work(id int) {
    fmt.Println("inicio", id)
    compute()
    fmt.Println("fin", id)
}

func main() {
    go work(1)
    go work(2)
    work(0)
}
```

`go f()` crea una goroutine y continúa sin esperar su resultado. Al retornar `main`, termina el proceso y todas las goroutines.

Green threads: modelo M:N

$$G_1 \quad G_2 \quad G_3 \quad \cdots \quad G_M$$

$$\Downarrow \text{ scheduler del runtime } \Downarrow$$

$$T_1 \quad T_2 \quad \cdots \quad T_N$$

M goroutines se multiplexan sobre *N* threads del SO.

Implicancias para el programa

- El orden de ejecución **no está garantizado.**
- `go f()` no espera ni ofrece un `join`.
- **Comparten memoria**: pueden aparecer data races.
- Sincronización y finalización explícitas.

Requiere correctitud para cualquier interleaving válido.

Channels

```go
type Message struct {
    Text string
}

messages := make(chan Message)

go func() {
    messages <- Message{Text: "hola"}
}()

msg := <-messages
```

Canal tipado

`chan Message` transporta solamente valores de tipo `Message`.

Operaciones bloqueantes

`ch <- value`: espera hasta poder enviar.

`value := <-ch`: espera hasta poder recibir.

Dos funciones

Comunicar un valor y establecer un punto de sincronización.

Victoria Canalizada

Victoria Canalizada

Victoria se mudó a San Martín de los Andes, así que sus colaboradores ya no pueden acercarse al taller.

Le envían los paquetes por **Correo Argentino**, con las mismas reglas que el problema original: una carrocería y 2 faros, *lo justo*.

Los colaboradores esperan a que Victoria termine el auto antes de mandar más.



Famosa marca Italiana

Generación Paralela

Generación Paralela

Cayó una auditoría en la materia y no pasamos la inspección. Por suerte, el inspector nos indicó que si contribuimos a su ONG, podía reconsiderar su postura.

Imbuido de una recientemente adquirida sabiduría china, Hernán nos propuso un novedoso sistema de inversión: periódicamente hacés una contribución monetaria a la persona que te invitó. A cambio, podés invitar a otras **4 personas** más. Recibirás comisiones por cada envío de dinero, multiplicando tus ganancias y *todos ganando en el proceso*.

Participar es obligatorio para presentarse al final y la nota es proporcional a la cantidad de familiares y amigos que invites.



Novedosa propuesta de inversión

Unbuffered channels

```go
func produce(out chan string) {
    fmt.Println("P: antes")
    out <- "dato"
    fmt.Println("P: despues")
}

func main() {
    ch := make(chan string)
    go produce(ch)

    fmt.Println("C: esperando")
    msg := <-ch
    fmt.Println("C:", msg)
}
```



Encuentro entre goroutines

Productor: `out <- "dato"`

⇓ espera al consumidor ⇓

Consumidor: `msg := <-ch`

Cuando se encuentran

El valor se transfiere y ambas goroutines pueden continuar.

Capacidad cero: no hay valores pendientes.

Streams: dirección, cierre e iteración

```go
func produce() <-chan int {
    out := make(chan int)

    go func() {
        defer close(out)
        for i := 1; i <= 3; i++ {
            out <- i
        }
    }()

    return out
}
```

`chan<- T`: solo envío

`<-chan T`: solo recepción

```go
values := produce()

for value := range values {
    fmt.Println(value)
}

value, ok := <-values
// value == 0, ok == false
```

`close` es una señal

Indica que no habrá más valores. Los valores buffered pendientes se reciben antes de terminar el `range`.

Cierra quien produce, intentar enviar luego del cierre produce panic.



Parte I: los inversores

- Hay un array con los nombres de los inversores. Hagan una goroutine que exponga todos los nombres en un canal, **numerados** para evitar repeticiones.
- Pueden usar `fmt.Sprintf("(%d)%s", i, nombre)` para numerarlos.
- En `main`, completen la función `invitar`: cada asociado invita hasta **4 subordinados**, y así sucesivamente, *hasta que no haya más personas en la lista*.

Wait Groups: esperar a que todas las goroutines terminen

```go
func work(id int,
    wg *sync.WaitGroup) {
    defer wg.Done()
    maybeHalt(id)
}

func main() {
    var wg sync.WaitGroup

    for id := 0; id < 3; id++ {
        wg.Add(1)
        go work(id, &wg)
    }

    wg.Wait()
    fmt.Println("Todos terminaron")
}
```

Un contador de tareas pendientes

`Add(1)`: registra una tarea.

`Done()`: indica que una tarea terminó.

`Wait()`: bloquea hasta llegar a cero.

Qué garantiza

- Después de `Wait()`, todas las tareas registradas hicieron `Done()`.
- El orden de finalización **no está garantizado**.

`TL;DR: Add` antes de `go` y `defer Done()` al entrar.

Compartir el mismo `WaitGroup` por puntero.



Parte II: la pirámide

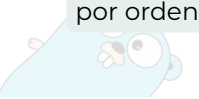
- Usen wait groups (`sync.WaitGroup`) para que ahora sí se puedan generar **todos** los inversores posibles.

`select`: esperar múltiples eventos

```
select {
case job := <-urgent:
    handle(job)
case job := <-normal:
    handle(job)
case <-time.After(time.Second):
    // time.After(...) sends the timestamp
    // when the timer fires
    fmt.Println("zzz")
}
```

Selección

Espera hasta que algún caso pueda avanzar. Si hay varios listos, elige uno **sin prioridad** por orden en la expresión.



```
select {
case value := <-results:
    use(value)
default:
    doOtherWork()
}
```

`default`

Se ejecuta inmediatamente si ningún otro caso está listo: convierte el `select` en no bloqueante.

Una selección

`select` elige una vez; para repetir debe estar dentro de un loop.

Worker pool: productor–consumidores

```go
func worker(
    id int,
    jobs <-chan Job,
    results chan<- Result,
) {
    for job := range jobs {
        results <- process(job)
    }
}
```

Cada worker consume hasta que `jobs` se cierra.

```go
jobs := make(chan Job)
results := make(chan Result)

for id := 0; id < 3; id++ {
    go worker(id, jobs, results)
}

go func() {
    defer close(jobs)
    for _, job := range input {
        jobs <- job
    }
}()

for range input {
    fmt.Println(<-results)
}
```



`input` → `jobs` → workers → `results` → `main`

La topología define trabajo, backpressure y terminación.

Buffer: cola acotada y backpressure

```
1 jobs := make(chan Job, 2)
2
3 jobs <- jobA // no bloquea
4 jobs <- jobB // no bloquea
5 jobs <- jobC // bloquea: lleno
```

Creación

`make(chan T)`: capacidad cero.

`make(chan T, N)`: hasta `N` valores pendientes.

Estado del buffer: capacidad 2

jobA	jobB

`jobC` → espera

- Envío bloquea cuando el buffer está lleno.
- Recepción bloquea cuando está vacío.
- Los valores salen en orden FIFO.
- El productor puede adelantarse hasta la capacidad.

No es “asíncrono”

El buffer absorbe ráfagas, pero conserva backpressure.

Channels: errores frecuentes

Deadlock

```
ch := make(chan int)
ch <- 1 // nadie recibe
```

Canal cerrado

```
close(ch)
ch <- 2 // panic
```

- Un `range` no termina si nadie cierra el canal.
- Enviar o recibir sobre un canal `nil` bloquea siempre.
- Un consumidor que abandona puede dejar al productor bloqueado.
- Agregar buffer no corrige un protocolo incorrecto.

Pregunta de diseño

¿Quién produce, quién consume, quién cierra y cómo termina cada goroutine?

Todo canal necesita propiedad clara; toda goroutine, una condición de salida.



Parte III: Recaudación

Ahora sí, toca hacer el sistema de contribuciones. Funciona así:

- Todos los inversores arrancan con **1000 yuanes**.
- Si tengo asociados subordinados:
  - Si *no* tengo jefe, me lo quedo.
  - Si tengo jefe, le reenvío los pagos.
- Si me quedé sin asociados subordinados:
  - Cada **100 ms**, envío **100 yuanes**.
  - Si tengo menos de 100 yuanes, envío un pago de 0 (que indica que me cierro) y retorno.
  - Si soy el jefe principal (no tengo jefe), en cuanto me quedo sin subordinados **imprimo el total obtenido** y retorno.



Parte IV: Teoría del Derrame

Finalmente, queda hacer el sistema de comisiones, el que permite volvernos ricos a todos. Funciona así:

- Al recibir pagos, el jefe principal (el que no tiene jefe) se queda con el **50 %** y envía el restante a sus subordinados por partes iguales.
- Si recibo una comisión de mi jefe, me quedo con el **50 %** y envío el restante en partes iguales a mis subordinados.
- Si recibo una comisión y *no* tengo subordinados, me quedo con el **total**.